DirectX 12 & DXR Path Tracer Engine Architecture Guide

CustomEngine 소스 코드 전체 심층 해설서

DirectX 12 입문자를 위한 파트별 완벽 구현 가이드 & 코드 워크스루

언어 / 표준: C++20 (MSVC /W4), HLSL SM 6.3/6.5 **그래픽스 API:** DirectX 12 Core, DXR Tier 1.1 **문서 버전:** v1.0.0 (Report Edition)

> 📌 **본 문서의 목적과 독자 안내**
>
> 이 문서는 CustomEngine의 전체 소스 코드(Win32 창 생성, 카메라 수학, DXC 셰이더 컴파일러, 지오메트리 버퍼, DXR 2단계 가속 구조, SBT 메모리 정렬, HLSL 패스 트레이싱, 프레임 렌더 루프 및 동기화)를 **DirectX 12를 처음 접하는 초심자의 눈높이**에서 한 줄 한 줄 원리와 배경을 명쾌하게 해설합니다. 모든 코드는 IDE(통합개발환경)에서 보는 것과 동일하게 구문 강조(Syntax Highlighting) 및 깨짐 방지 래핑이 적용되어 있습니다.

Part 1 DirectX 12 핵심 아키텍처와 개념 기초

DirectX 11 이전의 그래픽스 API는 드라이버가 배후에서 수많은 일(메모리 관리, 스레드 동기화, 하드웨어 파이프라인 상태 유추)을 대신 처리해 주는 *블랙박스(Black Box)* 형태였습니다. 반면 **DirectX 12**는 드라이버 오버헤드를 극도로 줄이고 개발자가 GPU의 하드웨어를 직접 제어할 수 있도록 완전히 재설계된 *명시적(Explicit) 로우레벨 API*입니다.

1.1 DirectX 12의 7대 핵심 객체

DirectX 12로 렌더링을 수행하기 위해서는 아래의 7대 핵심 개념과 객체를 반드시 이해해야 합니다:

핵심 객체 / 인터페이스	역할 및 설명	초보자를 위한 비유
Device (`ID3D12Device5`)	가상 그래픽 어댑터. 리소스 생성, 파이프라인 생성, DXR 지원 검사 등 모든 객체 할당의 공장(Factory) 역할.	그래픽 카드를 통제하는 총사령부
Command Queue / Allocator / List	명령 리스트(Command List)에 그리기 명령들을 기록하고, 메모리 풀인 할당자(Allocator)를 거쳐, 실제 GPU 큐(Command Queue)로 전송.	작업 지시서(List)를 결재판(Allocator)에 모아 GPU 작업라인(Queue)에 투입
Swap Chain (`IDXGISwapChain3`)	화면에 렌더링 결과물을 표시하기 위한 더블/트리플 버퍼 교환 장치. 티어링 방지 및 프레젠테이션.	앞면 도화지에 그림을 보여주는 동안 뒷면 도화지에 다음 프레임을 그리는 교환대
Descriptor Heap (서술자 힙)	텍스처, 버퍼, 상수 버퍼를 셰이더가 참조할 수 있도록 뷰(View)들의 메타데이터 포인터를 담는 GPU 메모리 배열.	셰이더가 볼 수 있는 책들의 인덱스 목차 카드함
Root Signature (루트 시그니처)	셰이더 레지스터(b0, t0, u0 등)와 서술자 힙/상수 버퍼의 매핑 관계를 선언하는 계약서.	함수의 매개변수 선언부 (어떤 인자가 어디로 전달되는지 명시)
Pipeline State (PSO / State Object)	셰이더, 블렌딩, 래스터라이저, 깊이/스텐실 상태를 하나로 결합하여 하드웨어에 고정(Immutable)한 상태 블록.	공장 조립 라인의 완전 세팅된 금형 틀
Fence & Resource Barrier	CPU-GPU 동기화를 위한 카운터(Fence)와, 자원의 상태(읽기/쓰기/복사/출력) 전이를 GPU에 알리는 안전장치(Barrier).	충돌을 막는 신호등과 공정 전환 알림판

1.2 DXR (DirectX Raytracing)과 2단계 가속 구조

기존 래스터라이제이션(Rasterization)이 3차원 삼각형들을 화면 2차원 평면에 투영(Projection)하여 픽셀을 채우는 방식이었다면, **DXR(DirectX Raytracing)**은 카메라에서 화면의 각 픽셀을 향해 가상의 광선(Ray)을 쏘고, 광선이 3D 월드의 삼각형과 교차(Intersection)하는지 수학적으로 검사합니다.

```
[Top-Level Acceleration Structure (TLAS)]  <-- 월드 내 객체들의 인스턴스 (변환 행렬, 마스크)
        |
        +--> Instance 0  ---> [Bottom-Level AS (BLAS 0)] ---> 메쉬 정점/인덱스 (Cube)
        +--> Instance 1  ---> [Bottom-Level AS (BLAS 1)] ---> 메쉬 정점/인덱스 (Sphere)
```

- **BLAS (Bottom-Level Acceleration Structure)**: 개별 3D 메쉬의 정점(Vertex)과 인덱스(Index) 기하 정보를 포함하는 트리 구조. 로컬 공간(Local Space) 기준.
- **TLAS (Top-Level Acceleration Structure)**: 월드 공간(World Space) 내에 배치된 BLAS들의 인스턴스(Instance) 목록과 3x4 아핀 변환 행렬 정보를 담는 최상위 트리.

Part 2 애플리케이션 진입점 및 윈도우 생성 (main & Window)

프로그램의 시작은 Win32 진입점 `wWinMain`에서 시작됩니다. 그래픽스 애플리케이션은 윈도우 OS의 이벤트 루프(Message Pump)와 긴밀하게 연동되어야 하며, 시간의 흐름(Delta Time)을 마이크로초 단위로 정밀하게 측정해야 합니다.

2.1 진입점 및 게임 루프 분석 (src/main.cpp)

src/main.cpp

```
#include "Window.h"
#include "DX12Renderer.h"
#include <memory>
#include <Windows.h>

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PWSTR pCmdLine, int nCmdShow)
{
    (void)hInstance; (void)hPrevInstance; (void)pCmdLine; (void)nCmdShow;
    LPWSTR fullCmdLine = GetCommandLineW();
    bool isSmokeTest = (fullCmdLine && (wcsstr(fullCmdLine, L"--smoke-test") != nullptr || wcsstr(fullC
mdLine, L"-smoke") != nullptr));
    int smokeFrameCount = 0;
    const int maxSmokeFrames = 5;

    try
    {
        Window window(1280, 720, L"Custom Engine - DX12 Path Tracer");
        std::unique_ptr<DX12Renderer> renderer = std::make_unique<DX12Renderer>(window.GetHWND(), windo
w.GetWidth(), window.GetHeight());

        LARGE_INTEGER frequency;
        QueryPerformanceFrequency(&frequency);
        LARGE_INTEGER lastTime;
        QueryPerformanceCounter(&lastTime);

        bool isRunning = true;
        while (isRunning)
        {
            if (!window.ProcessMessages())
            {
                isRunning = false;
                break;
            }

            LARGE_INTEGER currentTime;
            QueryPerformanceCounter(&currentTime);
            float deltaTime = static_cast<float>(currentTime.QuadPart - lastTime.QuadPart) / frequency.
QuadPart;
            lastTime = currentTime;

            renderer->Update(deltaTime);
            renderer->Render();

            if (isSmokeTest)
            {
                smokeFrameCount++;
                if (smokeFrameCount >= maxSmokeFrames)
                {
                    break;
                }
            }
        }
    }
    catch (const std::exception& e)
```

```
    {
        OutputDebugStringA(e.what());
        FILE* fp = nullptr;
        fopen_s(&fp, "smoke_error.txt", "w");
        if (fp) { fputs(e.what(), fp); fclose(fp); }
        return 1;
    }
    catch (...)
    {
        OutputDebugStringA("Unknown fatal exception occurred.\n");
        FILE* fp = nullptr;
        fopen_s(&fp, "smoke_error.txt", "w");
        if (fp) { fputs("Unknown fatal exception occurred.\n", fp); fclose(fp); }
        return 1;
    }

    return 0;
}
```

핵심 라인별 해설:

- **라인 9~12 (헤드리스 스모크 테스트 판정)**: `GetCommandLineW()`를 통해 명령줄 인자를 읽고, 만약 `--smoke-test`가 포함되어 있다면 5프레임만 렌더링하고 자동으로 정상 종료(Exit code 0)하도록 설계되었습니다. 이는 CI/CD 파이프라인이나 자동화 루프에서 GUI 창이 무한정 대기하는 것을 방지하는 핵심 기법입니다.
- **라인 19~23 (고해상도 타이머)**: `QueryPerformanceFrequency`와 `QueryPerformanceCounter`는 CPU 하드웨어의 고해상도 타임스탬프 카운터를 읽어옵니다. 표준 `timeGetTime`이나 `std::chrono`보다 오차가 적어 프레임 간 `deltaTime`을 매우 부드럽고 정확하게 계산할 수 있습니다.
- **라인 27~31 (논블로킹 메시지 처리)**: `window.ProcessMessages()`가 `false`를 반환(즉, `WM_QUIT` 수신)하면 렌더링 루프를 즉시 탈출합니다.
- **라인 38~39 (Update & Render 분리)**: 카메라 이동, 점프 물리 계산 등의 상태 갱신은 `Update(deltaTime)`에서, GPU 명령 기록 및 화면 출력은 `Render()`에서 명확히 분리하여 실행합니다.

2.2 Win32 창 클래스 및 윈도우 프로시저 (src/Window.h / .cpp)

src/Window.cpp (윈도우 초기화 & 등록)

```cpp
#include "Window.h"

Window::Window(int width, int height, const std::wstring& title)
    : m_width(width), m_height(height), m_title(title), m_windowClassName(L"CustomEngineWindowClass")
{
    m_hInstance = GetModuleHandle(nullptr);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.style = CS_OWNDC;
    wc.lpfnWndProc = WindowProc;
    wc.cbClsExtra = 0;
    wc.cbWndExtra = 0;
    wc.hInstance = m_hInstance;
    wc.hIcon = LoadIcon(nullptr, IDI_APPLICATION);
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wc.hbrBackground = nullptr;
    wc.lpszMenuName = nullptr;
    wc.lpszClassName = m_windowClassName.c_str();
    wc.hIconSm = LoadIcon(nullptr, IDI_APPLICATION);

    RegisterClassExW(&wc);

    // Adjust window size so the client area matches the requested width/height
    RECT wr = { 0, 0, m_width, m_height };
    AdjustWindowRect(&wr, WS_OVERLAPPEDWINDOW, FALSE);

    m_hwnd = CreateWindowExW(
        0,
        m_windowClassName.c_str(),
        m_title.c_str(),
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT,
        wr.right - wr.left,
        wr.bottom - wr.top,
        nullptr,
        nullptr,
        m_hInstance,
        this // Pass pointer to this class to use in WindowProc
    );

    ShowWindow(m_hwnd, SW_SHOW);
}

Window::~Window()
```

> 💡 **AdjustWindowRect의 중요성**
>
> 윈도우의 타이틀 바, 외곽선 테두리는 일정한 픽셀 두께를 가집니다. 만약 단순히 `CreateWindowExW`에 1280x720을 전달하면 실제 그림이 그려지는 클라이언트 영역(Client Area)은 1264x681 등으로 축소되어 렌더 타깃과 종횡비(Aspect Ratio) 왜곡이 발생합니다. `AdjustWindowRect(&wr, WS_OVERLAPPEDWINDOW, FALSE);`를 호출하면 원하는 클라이언트 크기가 정확히 나오도록 윈도우 전체 창 크기를 역계산해 줍니다.

src/Window.cpp (WindowProc 및 인스턴스 바인딩)

```
LRESULT CALLBACK Window::WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
    Window* pWindow = nullptr;

    if (uMsg == WM_NCCREATE)
    {
        CREATESTRUCT* pCreate = reinterpret_cast<CREATESTRUCT*>(lParam);
        pWindow = reinterpret_cast<Window*>(pCreate->lpCreateParams);
        SetWindowLongPtr(hwnd, GWLP_USERDATA, reinterpret_cast<LONG_PTR>(pWindow));
    }
    else
    {
        pWindow = reinterpret_cast<Window*>(GetWindowLongPtr(hwnd, GWLP_USERDATA));
    }

    if (pWindow)
    {
        switch (uMsg)
        {
        case WM_CLOSE:
            PostQuitMessage(0);
            return 0;
        }
    }

    return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
```

C++ 인스턴스 바인딩의 비밀: 정적(static) 함수인 `WindowProc`은 멤버 변수에 직접 접근할 수 없습니다. 따라서 창이 생성되는 최초 순간(`WM_NCCREATE`)에 `lParam`으로 전달된 `this` 포인터를 `SetWindowLongPtr(hwnd, GWLP_USERDATA, ...)`를 통해 Win32 윈도우 내부 메모리에 저장합니다. 이후 발생하는 모든 메시지는 `GetWindowLongPtr`을 통해 `Window*`를 복원하여 안전하게 처리합니다.

Part 3 DirectX 12 하드웨어 초기화 심층 분석 (InitDX12)

DirectX 12를 구동하기 위한 하드웨어 초기화 과정은 `DX12Renderer::InitDX12()`에 집약되어 있습니다. 이 함수는 디버그 레이어 켜기부터 그래픽 카드 선택, 명령 큐 생성, 스왑 체인 구축까지를 담당합니다.

3.1 디버그 레이어 및 DXR 하드웨어 어댑터 자동 탐색

src/DX12Renderer.cpp (디버그 레이어 & DXR 어댑터 탐색)

```
void DX12Renderer::InitDX12()
{
#if defined(_DEBUG)
    ComPtr<ID3D12Debug> debugController;
    if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&debugController))))
    {
        debugController->EnableDebugLayer();
    }
#endif

    UINT dxgiFactoryFlags = 0;
#if defined(_DEBUG)
    dxgiFactoryFlags |= DXGI_CREATE_FACTORY_DEBUG;
#endif
    ThrowIfFailed(CreateDXGIFactory2(dxgiFactoryFlags, IID_PPV_ARGS(&m_dxgiFactory)));

    ComPtr<IDXGIAdapter1> adapter;
    for (UINT i = 0; m_dxgiFactory->EnumAdapters1(i, &adapter) != DXGI_ERROR_NOT_FOUND; ++i)
    {
        DXGI_ADAPTER_DESC1 desc;
        adapter->GetDesc1(&desc);
        if (desc.Flags & DXGI_ADAPTER_FLAG_SOFTWARE)
        {
            continue;
        }

        ComPtr<ID3D12Device5> testDevice;
        if (SUCCEEDED(D3D12CreateDevice(adapter.Get(), D3D_FEATURE_LEVEL_12_1, IID_PPV_ARGS(&testDevic
e))))
        {
            D3D12_FEATURE_DATA_D3D12_OPTIONS5 featureSupportData = {};
            if (SUCCEEDED(testDevice->CheckFeatureSupport(D3D12_FEATURE_D3D12_OPTIONS5, &featureSupport
Data, sizeof(featureSupportData))) &&
                featureSupportData.RaytracingTier != D3D12_RAYTRACING_TIER_NOT_SUPPORTED)
            {
                m_device = testDevice;
                break;
            }
        }
    }

    if (!m_device)
    {
        throw std::runtime_error("No DXR-capable DirectX 12 hardware adapter found.");
    }
```

> ⚠️ **디버그 레이어(Debug Layer)는 입문자의 생명줄**
> DirectX 12 API 호출 실패는 기본적으로 아무런 크래시 메시지 없이 침묵하거나 GPU 크래시(TDR)를 일으킵니다. 디버그 모드에서 `ID3D12Debug::EnableDebugLayer()`를 활성화하면 잘못된 리소스 상태 전이, 잘못된 정렬, 바인딩 누락을 Visual Studio 디버그 출력 창에 상세한 영문 경고로 출력해 줍니다.

DXR 1.1 하드웨어 어댑터 필터링: 시스템에 인텔/AMD 내장 그래픽과 NVIDIA 외장 그래픽(RTX 시리즈)이 동시에 존재하는 경우가 많습니다. 코드는 `EnumAdapters1`을 순회하며 소프트웨어 어댑터(WARP)를 건너뛰고, `CheckFeatureSupport`로 `RaytracingTier != D3D12_RAYTRACING_TIER_NOT_SUPPORTED`인 하드웨어를 검출하여 `m_device`로 설정합니다. DXR을 사용하려면 반드시 `ID3D12Device5` 이상의 인터페이스가 필요합니다.

3.2 명령 큐, 스왑 체인 및 RTV 디스크립터 힙

src/DX12Renderer.cpp (명령 큐, 스왑 체인 & RTV 힙)

```
D3D12_COMMAND_QUEUE_DESC queueDesc = {};
queueDesc.Flags = D3D12_COMMAND_QUEUE_FLAG_NONE;
queueDesc.Type = D3D12_COMMAND_LIST_TYPE_DIRECT;

ThrowIfFailed(m_device->CreateCommandQueue(&queueDesc, IID_PPV_ARGS(&m_commandQueue)));

DXGI_SWAP_CHAIN_DESC1 swapChainDesc = {};
swapChainDesc.BufferCount = FrameCount;
swapChainDesc.Width = m_width;
swapChainDesc.Height = m_height;
swapChainDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_DISCARD;
swapChainDesc.SampleDesc.Count = 1;

ComPtr<IDXGISwapChain1> swapChain;
ThrowIfFailed(m_dxgiFactory->CreateSwapChainForHwnd(
    m_commandQueue.Get(),
    m_hwnd,
    &swapChainDesc,
    nullptr,
    nullptr,
    &swapChain
));

ThrowIfFailed(swapChain.As(&m_swapChain));
m_frameIndex = m_swapChain->GetCurrentBackBufferIndex();

D3D12_DESCRIPTOR_HEAP_DESC rtvHeapDesc = {};
rtvHeapDesc.NumDescriptors = FrameCount;
rtvHeapDesc.Type = D3D12_DESCRIPTOR_HEAP_TYPE_RTV;
rtvHeapDesc.Flags = D3D12_DESCRIPTOR_HEAP_FLAG_NONE;
ThrowIfFailed(m_device->CreateDescriptorHeap(&rtvHeapDesc, IID_PPV_ARGS(&m_rtvHeap)));
m_rtvDescriptorSize = m_device->GetDescriptorHandleIncrementSize(D3D12_DESCRIPTOR_HEAP_TYPE_RTV);

D3D12_CPU_DESCRIPTOR_HANDLE rtvHandle = m_rtvHeap->GetCPUDescriptorHandleForHeapStart();
for (UINT n = 0; n < FrameCount; n++)
{
    ThrowIfFailed(m_swapChain->GetBuffer(n, IID_PPV_ARGS(&m_renderTargets[n])));
    m_device->CreateRenderTargetView(m_renderTargets[n].Get(), nullptr, rtvHandle);
    rtvHandle.ptr += m_rtvDescriptorSize;
}
```

스왑 체인과 RTV 생성의 핵심 단계:

1. **`D3D12_COMMAND_QUEUE_DESC`**: 3D 렌더링 및 광선 추적 계산을 수행하는 기본 직접 실행형 큐(`D3D12_COMMAND_LIST_TYPE_DIRECT`)를 생성합니다.
2. **`DXGI_SWAP_CHAIN_DESC1`**: 화면 깜빡임과 렉을 방지하는 모던 프레젠테이션 모델인 `DXGI_SWAP_EFFECT_FLIP_DISCARD`를 설정합니다. 버퍼 개수는 2(더블 버퍼링)입니다.
3. **`D3D12_DESCRIPTOR_HEAP_DESC`**: RTV(Render Target View)를 위한 서술자 힙을 생성합니다. RTV 힙은 GPU 셰이더에서 직접 샘플링하는 용도가 아니므로 `D3D12_DESCRIPTOR_HEAP_FLAG_NONE`을 사용합니다.

4. **RTV 뷰 생성 루프**: `m_swapChain->GetBuffer(n, ...)`로 각 백버퍼의 `ID3D12Resource` 핸들을 얻고, `CreateRenderTargetView`를 호출하여 힙의 핸들에 순차적으로 바인딩합니다. 이때 다음 슬롯 주소는 `m_rtvDescriptorSize`만큼 포인터를 증가시켜 결정합니다.

Part 4 3D 수학, 카메라 및 물리 시뮬레이션 (Camera)

패스 트레이싱에서 카메라는 단순히 물체를 보는 관점이 아니라, **광선이 출발하는 원점(Origin)과 방향(Direction)을 결정하는 핵심 수학 엔진**입니다.

4.1 DirectXMath의 메모리 정렬과 벡터 연산 규칙

`DirectXMath`는 x86/x64의 SSE/AVX SIMD 레지스터(128비트)를 활용하여 4개의 부동소수점 연산을 한 번의 CPU 사이클에 수행합니다.

- `XMVECTOR` / `XMMATRIX`: SIMD 레지스터에 상주하는 16바이트 정렬 타입입니다. 구조체의 멤버 변수로 저장하면 메모리 정렬 위반으로 충돌할 수 있으므로, **지역 변수 연산용**으로만 사용합니다.
- `XMFLOAT3` / `XMFLOAT4X4`: 클래스 멤버 변수 저장용 타입입니다. 연산 시에는 `XMLoadFloat3`로 불러오고, 결과는 `XMStoreFloat3`로 다시 저장합니다.

4.2 오일러 각을 이용한 방향 벡터 갱신 (src/Camera.cpp)

src/Camera.cpp (UpdateVectors)

```
void Camera::UpdateVectors()
{
    float pitch = XMConvertToRadians(m_pitch);
    float yaw = XMConvertToRadians(m_yaw);

    XMMATRIX rotation = XMMatrixRotationRollPitchYaw(pitch, yaw, 0.0f);
    XMVECTOR forward = XMVector3TransformCoord(XMVectorSet(0.0f, 0.0f, 1.0f, 0.0f), rotation);
    XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);
    XMVECTOR right = XMVector3Cross(up, forward);
    up = XMVector3Cross(forward, right);

    XMStoreFloat3(&m_forward, XMVector3Normalize(forward));
    XMStoreFloat3(&m_right, XMVector3Normalize(right));
    XMStoreFloat3(&m_up, XMVector3Normalize(up));
}
```

카메라의 전방(Forward) 벡터는 기본 축 (0, 0, 1)에 회전 행렬을 곱하여 얻습니다. 이후 상단(Up, 0, 1, 0) 벡터와 전방 벡터의 **외적(Cross Product)**을 취해 우측(Right) 벡터를 구하고, 다시 외적을 취해 상호 직교(Orthogonal)하는 정규화 축들을 확립합니다.

4.3 1인칭 이동 및 점프 물리 시뮬레이션

src/Camera.cpp (UpdatePhysics)

```cpp
bool Camera::UpdatePhysics(float deltaTime)
{
    if (m_isGrounded && m_verticalVelocity == 0.0f)
    {
        return false;
    }

    float dt = (deltaTime > 0.1f) ? 0.1f : deltaTime;
    m_position.y += m_verticalVelocity * dt;
    m_verticalVelocity -= m_gravity * dt;

    if (m_position.y <= m_groundY)
    {
        m_position.y = m_groundY;
        m_verticalVelocity = 0.0f;
        m_isGrounded = true;
    }

    return true;
}
```

📐 XZ 평면 이동 투영 (Plane Projection)

카메라가 하늘이나 땅을 보고 있을 때(Pitch 각도가 클 때) 단순히 전방(Forward) 벡터로 이동하면 카메라가 공중으로 날아가거나 땅으로 파고듭니다. `Camera::Update()`에서는 전방 벡터의 Y 성분을 0으로 강제 투영(`XMVectorSet(m_forward.x, 0.0f, m_forward.z, 0.0f)`)하여 수평 이동을 보장합니다.

Part 5 모던 DXC 런타임 셰이더 컴파일러 (ShaderCompiler)

과거 DirectX 9~11 시절에는 `d3dcompiler_47.dll`의 `D3DCompile` API를 사용했습니다. 그러나 DXR과 최신 Shader Model 6.0 이상을 사용하기 위해서는 LLVM 오픈소스 기반의 모던 컴파일러인 **DirectX Shader Compiler (DXC)**가 필수적입니다.

5.1 DXC 컴파일러 아키텍처 및 구현 (src/ShaderCompiler.cpp)

src/ShaderCompiler.cpp

```
#include "ShaderCompiler.h"
#include <stdexcept>
#include <vector>
#include <iostream>
#include <Windows.h>

inline void ThrowIfFailed(HRESULT hr) {
    if (FAILED(hr)) throw std::runtime_error("DXC failed");
}

ShaderCompiler::ShaderCompiler() {
    ThrowIfFailed(DxcCreateInstance(CLSID_DxcUtils, IID_PPV_ARGS(&m_utils)));
    ThrowIfFailed(DxcCreateInstance(CLSID_DxcCompiler, IID_PPV_ARGS(&m_compiler)));
    ThrowIfFailed(m_utils->CreateDefaultIncludeHandler(&m_includeHandler));
}

ComPtr<IDxcBlob> ShaderCompiler::CompileShader(
    const std::wstring& filename,
    const std::wstring& entryPoint,
    const std::wstring& targetProfile)
{
    ComPtr<IDxcBlobEncoding> sourceBlob;
    HRESULT hr = m_utils->LoadFile(filename.c_str(), nullptr, &sourceBlob);
    if (FAILED(hr)) {
        throw std::runtime_error("Could not load shader file");
    }

    DxcBuffer sourceBuffer;
    sourceBuffer.Ptr = sourceBlob->GetBufferPointer();
    sourceBuffer.Size = sourceBlob->GetBufferSize();
    sourceBuffer.Encoding = DXC_CP_ACP;

    std::vector<LPCWSTR> arguments = {
        filename.c_str(),
        L"-E", entryPoint.c_str(),
        L"-T", targetProfile.c_str(),
        L"-Zi", // Debug info
        L"-Qembed_debug", // Embed debug info into the shader
        L"-Od", // Disable optimization
    };

    ComPtr<IDxcResult> results;
    hr = m_compiler->Compile(
        &sourceBuffer,
        arguments.data(),
        (UINT32)arguments.size(),
        m_includeHandler.Get(),
        IID_PPV_ARGS(&results)
    );
    ThrowIfFailed(hr);

    ComPtr<IDxcBlobUtf8> errors;
    results->GetOutput(DXC_OUT_ERRORS, IID_PPV_ARGS(&errors), nullptr);
    if (errors != nullptr && errors->GetStringLength() != 0) {
```

```
        OutputDebugStringA(errors->GetStringPointer());
        std::cerr << errors->GetStringPointer() << std::endl;
        throw std::runtime_error("Shader compilation failed");
    }

    HRESULT status;
    results->GetStatus(&status);
    ThrowIfFailed(status);

    ComPtr<IDxcBlob> shader;
    ThrowIfFailed(results->GetOutput(DXC_OUT_OBJECT, IID_PPV_ARGS(&shader), nullptr));

    return shader;
}
```

핵심 컴파일 인자 플래그 해설:

- `-E entryPoint` : 진입점 함수 이름 지정. DXR 라이브러리(`lib_6_3`) 컴파일 시에는 개별 진입점이 없으므로 공백(`L""`)을 전달합니다.
- `-T targetProfile` : 대상 셰이더 프로파일. 레이트레이싱은 `lib_6_3` 또는 `lib_6_5` , 버텍스 셰이더는 `vs_6_0` 등을 지정합니다.
- `-Zi` & `-Qembed_debug` : 컴파일된 DXIL 바이너리 내부에 PDB 디버깅 정보를 직접 삽입합니다. PIX나 RenderDoc 같은 GPU 디버거로 프레임을 캡처할 때 셰이더 원본 소스 코드를 라인 단위로 볼 수 있게 해줍니다.
- `-Od` : 컴파일러의 코드 최적화를 꺼서 디버깅 시 변수 값이 보존되도록 합니다.
- `DXC_OUT_ERRORS` : 문법 오류 발생 시 문자열 버퍼를 추출하여 콘솔 및 `OutputDebugStringA` 로 출력한 후 C++ 예외를 발생시킵니다.

Part 6 3D 모델 로딩 및 지오메트리 버퍼 생성 (Geometry Buffers)

GPU 렌더링의 기본 단위는 정점(Vertex)과 인덱스(Index)입니다. CPU 메모리에 있는 지오메트리 데이터를 고속의 GPU 전용 VRAM으로 옮기기 위해서는 **기본 버퍼 복사 패턴(Default Buffer Copy Pattern)**을 사용해야 합니다.

6.1 고속 VRAM 버퍼 생성 패턴 (CreateDefaultBuffer)

src/DX12Renderer.cpp (CreateDefaultBuffer)

```
ComPtr<ID3D12Resource> DX12Renderer::CreateDefaultBuffer(const void* data, UINT64 byteSize, ComPtr<ID3D
12Resource>& uploadBuffer)
{
    ComPtr<ID3D12Resource> defaultBuffer;

    D3D12_HEAP_PROPERTIES heapPropsDefault = {};
    heapPropsDefault.Type = D3D12_HEAP_TYPE_DEFAULT;

    D3D12_RESOURCE_DESC bufferDesc = {};
    bufferDesc.Dimension = D3D12_RESOURCE_DIMENSION_BUFFER;
    bufferDesc.Alignment = 0;
    bufferDesc.Width = byteSize;
    bufferDesc.Height = 1;
    bufferDesc.DepthOrArraySize = 1;
    bufferDesc.MipLevels = 1;
    bufferDesc.Format = DXGI_FORMAT_UNKNOWN;
    bufferDesc.SampleDesc.Count = 1;
    bufferDesc.SampleDesc.Quality = 0;
    bufferDesc.Layout = D3D12_TEXTURE_LAYOUT_ROW_MAJOR;
    bufferDesc.Flags = D3D12_RESOURCE_FLAG_NONE;

    ThrowIfFailed(m_device->CreateCommittedResource(
        &heapPropsDefault,
        D3D12_HEAP_FLAG_NONE,
        &bufferDesc,
        D3D12_RESOURCE_STATE_COPY_DEST,
        nullptr,
        IID_PPV_ARGS(&defaultBuffer)));

    D3D12_HEAP_PROPERTIES heapPropsUpload = {};
    heapPropsUpload.Type = D3D12_HEAP_TYPE_UPLOAD;

    ThrowIfFailed(m_device->CreateCommittedResource(
        &heapPropsUpload,
        D3D12_HEAP_FLAG_NONE,
        &bufferDesc,
        D3D12_RESOURCE_STATE_GENERIC_READ,
        nullptr,
        IID_PPV_ARGS(&uploadBuffer)));

    UINT8* pData;
    D3D12_RANGE readRange = { 0, 0 };
    ThrowIfFailed(uploadBuffer->Map(0, &readRange, reinterpret_cast<void**>(&pData)));
    memcpy(pData, data, byteSize);
    uploadBuffer->Unmap(0, nullptr);

    m_commandList->CopyBufferRegion(defaultBuffer.Get(), 0, uploadBuffer.Get(), 0, byteSize);

    D3D12_RESOURCE_BARRIER barrier = {};
    barrier.Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    barrier.Transition.pResource = defaultBuffer.Get();
    barrier.Transition.StateBefore = D3D12_RESOURCE_STATE_COPY_DEST;
    barrier.Transition.StateAfter = D3D12_RESOURCE_STATE_NON_PIXEL_SHADER_RESOURCE;
    m_commandList->ResourceBarrier(1, &barrier);
```

```
    return defaultBuffer;
}
```

```
[CPU 시스템 메모리]  --- memcpy --->  [UPLOAD 힙 (중간 버퍼)]
                                                |
                                    CopyBufferRegion (GPU 복사)
                                                v
[DEFAULT 힙 (초고속 VRAM)]  <--- ResourceBarrier (COPY_DEST -> NON_PIXEL_SRV)
```

왜 업로드 버퍼를 거쳐서 복사해야 하는가?

GPU는 `D3D12_HEAP_TYPE_DEFAULT`로 할당된 비디오 메모리(VRAM)에 가장 빠르고 효율적으로 접근할 수 있습니다. 하지만 CPU는 이 메모리에 직접 쓸 수 없습니다. 따라서 CPU가 쓸 수 있는 `D3D12_HEAP_TYPE_UPLOAD` 임시 버퍼에 먼저 복사한 뒤, GPU 명령 리스트의 `CopyBufferRegion` 명령을 통해 초고속 복사를 수행하고 리소스 장벽을 통해 읽기 가능 상태로 전환합니다.

6.2 Wavefront OBJ 파싱 및 트라이앵글 분할 (LoadModel)

src/DX12Renderer.cpp (LoadModel)

```
void DX12Renderer::LoadModel()
{
    std::vector<XMFLOAT3> temp_vertices;
    std::vector<Vertex> vertices;
    std::vector<uint32_t> indices;

    std::ifstream file("assets/cube.obj");
    if (!file.is_open()) {
        throw std::runtime_error("Could not open assets/cube.obj");
    }

    std::string line;
    while (std::getline(file, line)) {
        std::istringstream iss(line);
        std::string type;
        iss >> type;

        if (type == "v") {
            XMFLOAT3 v;
            iss >> v.x >> v.y >> v.z;
            temp_vertices.push_back(v);
        } else if (type == "f") {
            for (int i = 0; i < 4; ++i) { // Our cube has quads
                std::string vertexData;
                iss >> vertexData;
                if (vertexData.empty()) break;

                std::istringstream viss(vertexData);
                std::string vIdxStr;
                std::getline(viss, vIdxStr, '/'); // Get vertex index

                uint32_t vIdx = std::stoi(vIdxStr) - 1;

                Vertex vertex = {};
                vertex.position = temp_vertices[vIdx];
                vertex.color = { vertex.position.x + 0.5f, vertex.position.y + 0.5f, vertex.position.z
+ 0.5f, 1.0f };

                vertices.push_back(vertex);
            }

            // Triangulate the quad (0 1 2 3 -> 0 1 2, 0 2 3)
            uint32_t baseIdx = static_cast<uint32_t>(vertices.size()) - 4;
            indices.push_back(baseIdx + 0);
            indices.push_back(baseIdx + 1);
            indices.push_back(baseIdx + 2);
            indices.push_back(baseIdx + 0);
            indices.push_back(baseIdx + 2);
            indices.push_back(baseIdx + 3);
        }
    }

    m_indexCount = static_cast<UINT>(indices.size());
    const UINT vertexBufferSize = static_cast<UINT>(vertices.size() * sizeof(Vertex));
```

```
    const UINT indexBufferSize = static_cast<UINT>(indices.size() * sizeof(uint32_t));

    m_vertexBuffer = CreateDefaultBuffer(vertices.data(), vertexBufferSize, m_vertexBufferUpload);
    m_vertexBufferView.BufferLocation = m_vertexBuffer->GetGPUVirtualAddress();
    m_vertexBufferView.StrideInBytes = sizeof(Vertex);
    m_vertexBufferView.SizeInBytes = vertexBufferSize;

    m_indexBuffer = CreateDefaultBuffer(indices.data(), indexBufferSize, m_indexBufferUpload);
    m_indexBufferView.BufferLocation = m_indexBuffer->GetGPUVirtualAddress();
    m_indexBufferView.Format = DXGI_FORMAT_R32_UINT;
    m_indexBufferView.SizeInBytes = indexBufferSize;
}
```

3D 모델 파일(`assets/cube.obj`)의 면(Face, `f`) 데이터는 사각형(Quad, 정점 4개: 0 1 2 3)으로 구성되어 있습니다. GPU의 DXR 하드웨어는 기본적으로 삼각형만 지원하므로, 인덱스를 `0-1-2`와 `0-2-3` 두 개의 삼각형으로 분할(Triangulation)하여 인덱스 버퍼를 구축합니다.

Part 7 DXR 2단계 가속 구조 (BLAS & TLAS) 완전 정복

DXR의 심장인 가속 구조(Acceleration Structure)를 구축하는 과정입니다. 이 단계에서 GPU 내부의 RT Core가 활용할 수 있는 최적화된 공간 분할 트리(BVH)가 생성됩니다.

7.1 BLAS 구축 (Bottom-Level Acceleration Structure)

src/DX12Renderer.cpp (BLAS 정의 및 Prebuild)

```
    D3D12_RAYTRACING_GEOMETRY_DESC geometryDesc = {};
    geometryDesc.Type = D3D12_RAYTRACING_GEOMETRY_TYPE_TRIANGLES;
    geometryDesc.Triangles.IndexBuffer = m_indexBuffer->GetGPUVirtualAddress();
    geometryDesc.Triangles.IndexCount = m_indexCount;
    geometryDesc.Triangles.IndexFormat = DXGI_FORMAT_R32_UINT;
    geometryDesc.Triangles.Transform3x4 = 0;
    geometryDesc.Triangles.VertexFormat = DXGI_FORMAT_R32G32B32_FLOAT;
    geometryDesc.Triangles.VertexCount = m_indexCount; // Just to be safe, vertexCount should be number
of vertices
    // Actually, m_vertexBufferView.SizeInBytes / sizeof(Vertex) is the correct vertex count.
    geometryDesc.Triangles.VertexCount = m_vertexBufferView.SizeInBytes / sizeof(Vertex);
    geometryDesc.Triangles.VertexBuffer.StartAddress = m_vertexBuffer->GetGPUVirtualAddress();
    geometryDesc.Triangles.VertexBuffer.StrideInBytes = sizeof(Vertex);
    geometryDesc.Flags = D3D12_RAYTRACING_GEOMETRY_FLAG_OPAQUE;

    D3D12_BUILD_RAYTRACING_ACCELERATION_STRUCTURE_INPUTS blasInputs = {};
    blasInputs.Type = D3D12_RAYTRACING_ACCELERATION_STRUCTURE_TYPE_BOTTOM_LEVEL;
    blasInputs.DescsLayout = D3D12_ELEMENTS_LAYOUT_ARRAY;
    blasInputs.pGeometryDescs = &geometryDesc;
    blasInputs.NumDescs = 1;
    blasInputs.Flags = D3D12_RAYTRACING_ACCELERATION_STRUCTURE_BUILD_FLAG_PREFER_FAST_TRACE;

    D3D12_RAYTRACING_ACCELERATION_STRUCTURE_PREBUILD_INFO blasPrebuildInfo = {};
    m_device->GetRaytracingAccelerationStructurePrebuildInfo(&blasInputs, &blasPrebuildInfo);
```

정점 버퍼의 시작 GPU 가상 주소(`VertexBuffer.StartAddress`)와 인덱스 버퍼 주소(`IndexBuffer`)를 `D3D12_RAYTRACING_GEOMETRY_DESC` 에 기술합니다. `GetRaytracingAccelerationStructurePrebuildInfo` 를 호출하면 드라이버가 BVH 빌드에 필요한 임시 메모리(`ScratchDataSizeInBytes`)와 최종 결과물 버퍼 크기(`ResultDataMaxSizeInBytes`)를 계산해 줍니다.

7.2 TLAS 구축 및 인스턴스 기술자 매핑

src/DX12Renderer.cpp (TLAS 인스턴스 및 빌드)

```
    // 5. Create Instance Descriptor for TLAS
    D3D12_RAYTRACING_INSTANCE_DESC instanceDesc = {};
    instanceDesc.Transform[0][0] = 1.0f; instanceDesc.Transform[0][1] = 0.0f; instanceDesc.Transform[0]
[2] = 0.0f; instanceDesc.Transform[0][3] = 0.0f;
    instanceDesc.Transform[1][0] = 0.0f; instanceDesc.Transform[1][1] = 1.0f; instanceDesc.Transform[1]
[2] = 0.0f; instanceDesc.Transform[1][3] = 0.0f;
    instanceDesc.Transform[2][0] = 0.0f; instanceDesc.Transform[2][1] = 0.0f; instanceDesc.Transform[2]
[2] = 1.0f; instanceDesc.Transform[2][3] = 0.0f;
    instanceDesc.InstanceID = 0;
    instanceDesc.InstanceMask = 0xFF;
    instanceDesc.InstanceContributionToHitGroupIndex = 0;
    instanceDesc.Flags = D3D12_RAYTRACING_INSTANCE_FLAG_NONE;
    instanceDesc.AccelerationStructure = m_bottomLevelAS->GetGPUVirtualAddress();

    D3D12_HEAP_PROPERTIES uploadHeap = {};
    uploadHeap.Type = D3D12_HEAP_TYPE_UPLOAD;
    D3D12_RESOURCE_DESC instanceDescDesc = scratchDesc;
    instanceDescDesc.Width = sizeof(instanceDesc);
    instanceDescDesc.Flags = D3D12_RESOURCE_FLAG_NONE;

    ThrowIfFailed(m_device->CreateCommittedResource(
        &uploadHeap,
        D3D12_HEAP_FLAG_NONE,
        &instanceDescDesc,
        D3D12_RESOURCE_STATE_GENERIC_READ,
        nullptr,
        IID_PPV_ARGS(&m_instanceDescs)));

    void* pMappedData;
    ThrowIfFailed(m_instanceDescs->Map(0, nullptr, &pMappedData));
    memcpy(pMappedData, &instanceDesc, sizeof(instanceDesc));
    m_instanceDescs->Unmap(0, nullptr);

    tlasInputs.InstanceDescs = m_instanceDescs->GetGPUVirtualAddress();

    // 6. Build BLAS
    D3D12_BUILD_RAYTRACING_ACCELERATION_STRUCTURE_DESC buildDesc = {};
    buildDesc.Inputs = blasInputs;
    buildDesc.ScratchAccelerationStructureData = m_scratchBuffer->GetGPUVirtualAddress();
    buildDesc.DestAccelerationStructureData = m_bottomLevelAS->GetGPUVirtualAddress();

    m_commandList->BuildRaytracingAccelerationStructure(&buildDesc, 0, nullptr);

    // Barrier to wait for BLAS build to finish
    D3D12_RESOURCE_BARRIER uavBarrier = {};
    uavBarrier.Type = D3D12_RESOURCE_BARRIER_TYPE_UAV;
    uavBarrier.UAV.pResource = m_bottomLevelAS.Get();
    m_commandList->ResourceBarrier(1, &uavBarrier);

    // 7. Build TLAS
    buildDesc.Inputs = tlasInputs;
    buildDesc.ScratchAccelerationStructureData = m_scratchBuffer->GetGPUVirtualAddress();
    buildDesc.DestAccelerationStructureData = m_topLevelAS->GetGPUVirtualAddress();
```

```
    m_commandList->BuildRaytracingAccelerationStructure(&buildDesc, 0, nullptr);

    // Barrier to wait for TLAS build to finish
    uavBarrier.UAV.pResource = m_topLevelAS.Get();
    m_commandList->ResourceBarrier(1, &uavBarrier);
}
```

핵심 구조체 분석: D3D12_RAYTRACING_INSTANCE_DESC

- `Transform[3][4]` : 3D 공간 상에서 객체의 위치, 회전, 크기를 나타내는 3x4 아핀 변환 행렬입니다. 여기서는 단위 행렬(Identity)이 적용되었습니다.
- `InstanceMask = 0xFF` : 광선 발사 시 특정 마스크와 비트 AND 연산을 수행하여 어떤 인스턴스를 무시할지 필터링할 수 있습니다. `0xFF` 는 모든 광선과 충돌합니다.
- `InstanceContributionToHitGroupIndex = 0` : 이 인스턴스에 광선이 충돌했을 때 SBT에서 참조할 히트 그룹(Hit Group)의 기본 오프셋입니다.
- `AccelerationStructure` : 대상 메쉬의 BLAS가 위치한 GPU 가상 주소(`m_bottomLevelAS->GetGPUVirtualAddress()`)를 지정합니다.

> **필수 동기화: UAV 리소스 장벽 (UAV Barrier)**
> BLAS 빌드와 TLAS 빌드는 모두 GPU 상에서 비동기로 실행됩니다. 만약 BLAS 빌드가 끝나기 전에 TLAS 빌드가 시작되면 깨진 지오메트리를 참조하여 크래시가 발생합니다. 코드의 `D3D12_RESOURCE_BARRIER_TYPE_UAV` 장벽은 앞선 빌드가 완전히 끝날 때까지 GPU 하드웨어를 안전하게 대기시킵니다.

Part 8 DXR 파이프라인(RTPSO) & 루트 시그니처

기존 래스터라이제이션이 단일 `ID3D12PipelineState`를 사용했다면, DXR은 여러 셰이더와 라이브러리가 유기적으로 연결된 거대한 상태 객체인 **State Object (RTPSO)**를 사용합니다.

8.1 글로벌 루트 시그니처 (Global Root Signature)

src/DX12Renderer.cpp (글로벌 루트 시그니처)

```
void DX12Renderer::LoadAssetsDXR()
{
    // 1. Create Global Root Signature
    D3D12_DESCRIPTOR_RANGE descriptorRanges[2] = {};

    // UAVs (u0 for RenderTarget, u1 for AccumulationBuffer)
    descriptorRanges[0].RangeType = D3D12_DESCRIPTOR_RANGE_TYPE_UAV;
    descriptorRanges[0].NumDescriptors = 2; // Two UAVs!
    descriptorRanges[0].BaseShaderRegister = 0;
    descriptorRanges[0].RegisterSpace = 0;
    descriptorRanges[0].OffsetInDescriptorsFromTableStart = D3D12_DESCRIPTOR_RANGE_OFFSET_APPEND;

    // SRV (t0) for Acceleration Structure
    descriptorRanges[1].RangeType = D3D12_DESCRIPTOR_RANGE_TYPE_SRV;
    descriptorRanges[1].NumDescriptors = 1;
    descriptorRanges[1].BaseShaderRegister = 0;
    descriptorRanges[1].RegisterSpace = 0;
    descriptorRanges[1].OffsetInDescriptorsFromTableStart = D3D12_DESCRIPTOR_RANGE_OFFSET_APPEND;

    D3D12_ROOT_PARAMETER rootParameters[5] = {};
    // RenderTarget + AccumulationBuffer (u0, u1)
    rootParameters[0].ParameterType = D3D12_ROOT_PARAMETER_TYPE_DESCRIPTOR_TABLE;
    rootParameters[0].DescriptorTable.NumDescriptorRanges = 1;
    rootParameters[0].DescriptorTable.pDescriptorRanges = &descriptorRanges[0];
    rootParameters[0].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

    // Scene (t0)
    rootParameters[1].ParameterType = D3D12_ROOT_PARAMETER_TYPE_SRV;
    rootParameters[1].Descriptor.ShaderRegister = 0;
    rootParameters[1].Descriptor.RegisterSpace = 0;
    rootParameters[1].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

    // Vertices (t1)
    rootParameters[2].ParameterType = D3D12_ROOT_PARAMETER_TYPE_SRV;
    rootParameters[2].Descriptor.ShaderRegister = 1;
    rootParameters[2].Descriptor.RegisterSpace = 0;
    rootParameters[2].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

    // Indices (t2)
    rootParameters[3].ParameterType = D3D12_ROOT_PARAMETER_TYPE_SRV;
    rootParameters[3].Descriptor.ShaderRegister = 2;
    rootParameters[3].Descriptor.RegisterSpace = 0;
    rootParameters[3].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

    // CameraCB (b0)
    rootParameters[4].ParameterType = D3D12_ROOT_PARAMETER_TYPE_CBV;
    rootParameters[4].Descriptor.ShaderRegister = 0;
    rootParameters[4].Descriptor.RegisterSpace = 0;
    rootParameters[4].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

    D3D12_ROOT_SIGNATURE_DESC rootSignatureDesc = {};
    rootSignatureDesc.NumParameters = 5;
    rootSignatureDesc.pParameters = rootParameters;
    rootSignatureDesc.NumStaticSamplers = 0;
```

```
    rootSignatureDesc.pStaticSamplers = nullptr;
    rootSignatureDesc.Flags = D3D12_ROOT_SIGNATURE_FLAG_NONE;

    ComPtr<ID3DBlob> signature;
    ComPtr<ID3DBlob> error;
    ThrowIfFailed(D3D12SerializeRootSignature(&rootSignatureDesc, D3D_ROOT_SIGNATURE_VERSION_1, &signat
ure, &error));
    ThrowIfFailed(m_device->CreateRootSignature(0, signature->GetBufferPointer(), signature->GetBufferS
ize(), IID_PPV_ARGS(&m_rootSignature)));
```

루트 파라미터는 셰이더에 데이터를 전달하는 통로입니다:

- **슬롯 0 (Descriptor Table)**: UAV 2개 (`u0` : 화면 출력 렌더 타깃, `u1` : 다중 프레임 색상 누적 버퍼).
- **슬롯 1 (Root SRV)**: TLAS 가속 구조 (`t0`).
- **슬롯 2 (Root SRV)**: 메쉬 정점 버퍼 구조화 버퍼 (`t1`).
- **슬롯 3 (Root SRV)**: 메쉬 인덱스 버퍼 구조화 버퍼 (`t2`).
- **슬롯 4 (Root CBV)**: 카메라 역행렬 및 프레임 번호를 담은 상수 버퍼 (`b0`).

8.2 DXR State Object 5대 하위 객체 (CreateRaytracingPipeline)

src/DX12Renderer.cpp (RTPSO 5대 서브오브젝트)

```
void DX12Renderer::CreateRaytracingPipeline()
{
    ComPtr<IDxcBlob> raytracingShader = m_shaderCompiler.CompileShader(L"shaders/raytracing.hlsl", L"",
L"lib_6_3");

    D3D12_STATE_OBJECT_DESC raytracingPipelineDesc = {};
    raytracingPipelineDesc.Type = D3D12_STATE_OBJECT_TYPE_RAYTRACING_PIPELINE;

    D3D12_STATE_SUBOBJECT subobjects[5] = {};

    // 1. DXIL Library
    D3D12_DXIL_LIBRARY_DESC libDesc = {};
    libDesc.DXILLibrary.pShaderBytecode = raytracingShader->GetBufferPointer();
    libDesc.DXILLibrary.BytecodeLength = raytracingShader->GetBufferSize();
    libDesc.NumExports = 0; // Export all
    libDesc.pExports = nullptr;

    subobjects[0].Type = D3D12_STATE_SUBOBJECT_TYPE_DXIL_LIBRARY;
    subobjects[0].pDesc = &libDesc;

    // 2. Hit Group
    D3D12_HIT_GROUP_DESC hitGroupDesc = {};
    hitGroupDesc.HitGroupExport = L"HitGroup";
    hitGroupDesc.Type = D3D12_HIT_GROUP_TYPE_TRIANGLES;
    hitGroupDesc.ClosestHitShaderImport = L"ClosestHit";
    hitGroupDesc.AnyHitShaderImport = nullptr;
    hitGroupDesc.IntersectionShaderImport = nullptr;

    subobjects[1].Type = D3D12_STATE_SUBOBJECT_TYPE_HIT_GROUP;
    subobjects[1].pDesc = &hitGroupDesc;

    // 3. Shader Config
    D3D12_RAYTRACING_SHADER_CONFIG shaderConfig = {};
    shaderConfig.MaxPayloadSizeInBytes = 10 * sizeof(float); // 40 bytes: float3 color (12) + float hit
T (4) + float3 normal (12) + float3 albedo (12)
    shaderConfig.MaxAttributeSizeInBytes = 2 * sizeof(float); // 8 bytes: float2 barycentrics

    subobjects[2].Type = D3D12_STATE_SUBOBJECT_TYPE_RAYTRACING_SHADER_CONFIG;
    subobjects[2].pDesc = &shaderConfig;

    // 4. Global Root Signature
    D3D12_GLOBAL_ROOT_SIGNATURE globalRootSig = {};
    globalRootSig.pGlobalRootSignature = m_rootSignature.Get();

    subobjects[3].Type = D3D12_STATE_SUBOBJECT_TYPE_GLOBAL_ROOT_SIGNATURE;
    subobjects[3].pDesc = &globalRootSig;

    // 5. Pipeline Config
    D3D12_RAYTRACING_PIPELINE_CONFIG pipelineConfig = {};
    pipelineConfig.MaxTraceRecursionDepth = 1; // 1 ray bounce (primary rays only)

    subobjects[4].Type = D3D12_STATE_SUBOBJECT_TYPE_RAYTRACING_PIPELINE_CONFIG;
    subobjects[4].pDesc = &pipelineConfig;
```

```
    raytracingPipelineDesc.NumSubobjects = 5;
    raytracingPipelineDesc.pSubobjects = subobjects;

    ThrowIfFailed(m_device->CreateStateObject(&raytracingPipelineDesc, IID_PPV_ARGS(&m_dxrStateObjec
t)));
}
```

> ⚠️ **DXR 입문자가 가장 많이 저지르는 버그 1위: 페이로드 바이트 크기 불일치**
>
> `shaderConfig.MaxPayloadSizeInBytes = 10 * sizeof(float);`는 정확히 40바이트입니다. HLSL의 `struct RayPayload { float3 color(12) + float hitT(4) + float3 normal(12) + float3 albedo(12) = 40 bytes }`와 단 1바이트라도 다르면 런타임에 `DXGI_ERROR_DEVICE_REMOVED` 크래시가 발생합니다!

Part 9 셰이더 바인딩 테이블 (SBT: Shader Binding Table)

광선이 허공을 날아갈 때(Miss), 물체에 부딪혔을 때(Closest Hit), 혹은 최초로 발사될 때(RayGen) 어떤 셰이더 코드를 실행해야 할까요? GPU는 CPU처럼 함수 포인터를 직접 호출할 수 없으므로, 메모리에 정렬된 테이블을 만들어 둡니다. 이것이 바로 **셰이더 바인딩 테이블(SBT)**입니다.

9.1 메모리 정렬 및 SBT 레이아웃 (src/DX12Renderer.cpp)

src/DX12Renderer.cpp (CreateShaderBindingTable)

```
void DX12Renderer::CreateShaderBindingTable()
{
    ComPtr<ID3D12StateObjectProperties> stateObjectProperties;
    ThrowIfFailed(m_dxrStateObject.As(&stateObjectProperties));

    void* rayGenShaderIdentifier = stateObjectProperties->GetShaderIdentifier(L"RayGen");
    void* missShaderIdentifier = stateObjectProperties->GetShaderIdentifier(L"Miss");
    void* hitGroupShaderIdentifier = stateObjectProperties->GetShaderIdentifier(L"HitGroup");

    m_sbtEntrySize = D3D12_SHADER_IDENTIFIER_SIZE_IN_BYTES;
    m_sbtEntrySize = (m_sbtEntrySize + (D3D12_RAYTRACING_SHADER_TABLE_BYTE_ALIGNMENT - 1)) & ~(D3D12_RA
YTRACING_SHADER_TABLE_BYTE_ALIGNMENT - 1); // Align to 64 bytes

    UINT sbtSize = m_sbtEntrySize * 3;

    D3D12_HEAP_PROPERTIES uploadHeap = {};
    uploadHeap.Type = D3D12_HEAP_TYPE_UPLOAD;

    D3D12_RESOURCE_DESC bufferDesc = {};
    bufferDesc.Dimension = D3D12_RESOURCE_DIMENSION_BUFFER;
    bufferDesc.Alignment = 0;
    bufferDesc.Width = sbtSize;
    bufferDesc.Height = 1;
    bufferDesc.DepthOrArraySize = 1;
    bufferDesc.MipLevels = 1;
    bufferDesc.Format = DXGI_FORMAT_UNKNOWN;
    bufferDesc.SampleDesc.Count = 1;
    bufferDesc.SampleDesc.Quality = 0;
    bufferDesc.Layout = D3D12_TEXTURE_LAYOUT_ROW_MAJOR;
    bufferDesc.Flags = D3D12_RESOURCE_FLAG_NONE;

    ThrowIfFailed(m_device->CreateCommittedResource(
        &uploadHeap,
        D3D12_HEAP_FLAG_NONE,
        &bufferDesc,
        D3D12_RESOURCE_STATE_GENERIC_READ,
        nullptr,
        IID_PPV_ARGS(&m_shaderBindingTable)));

    uint8_t* pData;
    ThrowIfFailed(m_shaderBindingTable->Map(0, nullptr, reinterpret_cast<void**>(&pData)));

    memcpy(pData, rayGenShaderIdentifier, D3D12_SHADER_IDENTIFIER_SIZE_IN_BYTES);
    memcpy(pData + m_sbtEntrySize, missShaderIdentifier, D3D12_SHADER_IDENTIFIER_SIZE_IN_BYTES);
    memcpy(pData + m_sbtEntrySize * 2, hitGroupShaderIdentifier, D3D12_SHADER_IDENTIFIER_SIZE_IN_BYTE
S);

    m_shaderBindingTable->Unmap(0, nullptr);
}
```

```
[Shader Binding Table (SBT) 메모리 레이아웃 (단일 UPLOAD 버퍼)]
+-----------------------+-----------------------+-----------------------+
| Entry 0: RayGen Record | Entry 1: Miss Record   | Entry 2: HitGroup Rec  |
| (Offset: 0B)           | (Offset: 64B)          | (Offset: 128B)         |
+-----------------------+-----------------------+-----------------------+
| 32B Shader Identifier  | 32B Shader Identifier  | 32B Shader Identifier  |
| 32B Padding            | 32B Padding            | 32B Padding            |
+-----------------------+-----------------------+-----------------------+
```

하드웨어 정렬 공식:

DirectX 12 스펙상 각 셰이더 레코드는 최소 32바이트(`D3D12_RAYTRACING_SHADER_RECORD_BYTE_ALIGNMENT`) 단위여야 하며, 개별 셰이더 테이블의 시작 주소는 64바이트(`D3D12_RAYTRACING_SHADER_TABLE_BYTE_ALIGNMENT`) 정렬이어야 합니다. 코드의 다음 비트 연산이 이를 완벽히 보장합니다:

`m_sbtEntrySize = (m_sbtEntrySize + (64 - 1)) & ~(64 - 1); // 32 -> 64바이트로 올림 정렬`

Part 10 DXR 패스 트레이싱 HLSL 셰이더 완전 정복

이제 모든 수학과 알고리즘이 실행되는 셰이더 코드(`shaders/raytracing.hlsl`)를 분석합니다. 이 셰이더는 단순한 단일 광선 검출을 넘어, 물리 기반의 다중 바운스 몬테카를로 경로 추적(Monte Carlo Path Tracing)과 안티에일리어싱, 시간적 누적 버퍼, 톤매핑을 모두 수행합니다.

10.1 PCG 난수 생성기 및 반구 코사인 샘플링

shaders/raytracing.hlsl (PCG 난수 & 반구 코사인 샘플링)

```
// PCG Random Number Generator
uint pcg_hash(uint input)
{
    uint state = input * 747796405u + 2891336453u;
    uint word = ((state >> ((state >> 28u) + 4u)) ^ state) * 277803737u;
    return (word >> 22u) ^ word;
}

float RandomFloat(inout uint seed)
{
    seed = pcg_hash(seed);
    return (float)seed / 4294967296.0;
}

float3 RandomCosineDirection(float3 n, inout uint seed)
{
    float u1 = RandomFloat(seed);
    float u2 = RandomFloat(seed);
    float r = sqrt(u1);
    float theta = 2.0 * 3.14159265 * u2;

    float x = r * cos(theta);
    float y = r * sin(theta);
    float z = sqrt(max(0.0, 1.0 - u1));

    float3 up = abs(n.z) < 0.999 ? float3(0, 0, 1) : float3(1, 0, 0);
    float3 tangent = normalize(cross(up, n));
    float3 bitangent = cross(n, tangent);

    return tangent * x + bitangent * y + n * z;
}
```

반구 코사인 샘플링(Cosine-weighted Hemisphere Sampling):
빛이 거친 표면(디퓨즈/난반사)에 부딪히면 램버트 코사인 법칙(cos θ)에 비례하여 모든 방향으로 산란됩니다. 균일(Uniform)하게 무작위 방향을 뽑으면 샘플링 효율이 떨어지므로, 표면 법선(N) 축을 중심으로 코사인 가중치가 적용된 X, Y, Z 방향 벡터를 구면 좌표계로부터 유도합니다.

10.2 RayGen 셰이더: 4회 바운스 패스 트레이싱 & 시간적 누적

shaders/raytracing.hlsl (RayGen 4회 바운스 & 누적)

```
[shader("raygeneration")]
void RayGen()
{
    uint2 launchIndex = DispatchRaysIndex().xy;
    uint2 launchDim = DispatchRaysDimensions().xy;

    uint seed = launchIndex.x + launchIndex.y * launchDim.x + frameCount * 1973;

    float2 jitter = float2(RandomFloat(seed), RandomFloat(seed));
    float2 crd = float2(launchIndex.x + jitter.x, launchIndex.y + jitter.y);
    float2 dims = float2(launchDim.x, launchDim.y);

    float2 d = ((crd / dims) * 2.0f - 1.0f);
    d.y = -d.y;

    float4 target = mul(float4(d.x, d.y, 1, 1), projInverse);
    float3 rayDir = normalize(mul(float4(normalize(target.xyz / target.w), 0), viewInverse).xyz);
    float3 origin = mul(float4(0, 0, 0, 1), viewInverse).xyz;

    float3 throughput = float3(1, 1, 1);
    float3 radiance = float3(0, 0, 0);

    for (int bounce = 0; bounce < 4; bounce++) {
        RayDesc ray;
        ray.Origin = origin;
        ray.Direction = rayDir;
        ray.TMin = 0.001f;
        ray.TMax = 10000.0f;

        RayPayload payload;
        payload.hitT = -1.0f;

        TraceRay(Scene, RAY_FLAG_NONE, ~0, 0, 1, 0, ray, payload);

        if (payload.hitT < 0.0f) {
            radiance += throughput * payload.color;
            break;
        }

        float3 hitPos = origin + rayDir * payload.hitT;
        float3 normal = payload.normal;
        if (dot(normal, rayDir) > 0.0f) {
            normal = -normal;
        }

        float3 nextDir = RandomCosineDirection(normal, seed);

        origin = hitPos + normal * 0.001f;
        rayDir = nextDir;
        throughput *= payload.albedo;
    }

    float3 currentRadiance = radiance;
```

```
    if (frameCount > 1) {
        float3 prevRadiance = AccumulationTarget[launchIndex].xyz;
        currentRadiance = lerp(prevRadiance, currentRadiance, 1.0f / float(frameCount));
    }

    AccumulationTarget[launchIndex] = float4(currentRadiance, 1.0f);

    // Simple Reinhard Tonemapping
    float3 mapped = currentRadiance / (currentRadiance + float3(1.0, 1.0, 1.0));
    // Gamma correction
    mapped = pow(mapped, float3(1.0/2.2, 1.0/2.2, 1.0/2.2));

    RenderTarget[launchIndex] = float4(mapped, 1.0f);
}
```

RayGen의 5단계 파이프라인:

1. **서브픽셀 지터링 (Anti-Aliasing)**: 픽셀 중심 대신 픽셀 영역 내부의 무작위 위치(`jitter`)를 조준하여 계단 현상(Aliasing)을 제거합니다.
2. **역행렬을 통한 1차 광선 생성**: 화면 정규화 좌표(NDC)에 `projInverse`와 `viewInverse`를 곱하여 월드 공간상의 광선 원점과 방향을 복원합니다.
3. **4회 바운스 루프**: `TraceRay`를 호출하여 충돌 지점을 찾고, 충돌 시 해당 지점의 법선 기준으로 새로운 반사 광선을 발사합니다. 광선이 반사될 때마다 물체의 표면 색상(`payload.albedo`)이 스루풋(`throughput`)에 누적 곱해집니다.
4. **시간적 누적 (Temporal Accumulation)**: 패스 트레이싱은 노이즈가 많습니다. 카메라가 정지해 있을 때 매 프레임의 광선 결과를 평균 누적합니다: `C_new = lerp(C_prev, C_curr, 1.0f / float(frameCount))`
5. **라인하르트 톤매핑 & 감마 보정**: 물리적으로 계산된 HDR 색상을 LDR 모니터에 표현하기 위해 `C / (C + 1.0)` 연산을 수행하고, 모니터의 비선형 밝기 특성에 맞추어 감마(1.0 / 2.2) 보정을 적용합니다.

10.3 Miss & ClosestHit 셰이더

shaders/raytracing.hlsl (Miss & ClosestHit)

```
[shader("miss")]
void Miss(inout RayPayload payload)
{
    float t = 0.5f * (WorldRayDirection().y + 1.0f);
    payload.color = lerp(float3(1.0f, 1.0f, 1.0f), float3(0.5f, 0.7f, 1.0f), t) * 1.5f; // Sky intensit
y
    payload.hitT = -1.0f;
}

[shader("closesthit")]
void ClosestHit(inout RayPayload payload, in BuiltInTriangleIntersectionAttributes attr)
{
    payload.hitT = RayTCurrent();

    uint primitiveIndex = PrimitiveIndex();
    uint i0 = Indices[primitiveIndex * 3 + 0];
    uint i1 = Indices[primitiveIndex * 3 + 1];
    uint i2 = Indices[primitiveIndex * 3 + 2];

    float3 v0 = VertexBuffer[i0].position;
    float3 v1 = VertexBuffer[i1].position;
    float3 v2 = VertexBuffer[i2].position;

    payload.normal = normalize(cross(v1 - v0, v2 - v0));

    // Instead of taking the vertex color, let's use a nice diffuse color
    payload.albedo = float3(0.8f, 0.3f, 0.3f); // Red
}
```

`ClosestHit`에서는 DXR 내장 인트린식 함수인 `PrimitiveIndex()`를 통해 삼각형 인덱스를 읽고, 정점 버퍼에서 3개의 정점을 가져와 **벡터 외적(Cross Product)**으로 정확한 표면 법선(Normal)을 실시간으로 계산해 냅니다.

Part 11 프레임 렌더 루프 및 GPU 동기화 (Update & Render)

매 프레임마다 실행되는 `DX12Renderer::Update`와 `Render`는 CPU와 GPU의 완벽한 오케스트레이션을 보여줍니다.

11.1 Update 및 상수 버퍼 갱신

src/DX12Renderer.cpp (Update)

```
void DX12Renderer::Update(float deltaTime)
{
    if (m_camera.Update(deltaTime)) {
        m_accumulationFrames = 0;
    }
    m_accumulationFrames++;

    XMMATRIX view = m_camera.GetViewMatrix();
    XMMATRIX proj = m_camera.GetProjectionMatrix(static_cast<float>(m_width) / m_height);

    XMVECTOR det;
    m_cbvDataBegin->viewInverse = XMMatrixTranspose(XMMatrixInverse(&det, view));
    m_cbvDataBegin->projInverse = XMMatrixTranspose(XMMatrixInverse(&det, proj));
    m_cbvDataBegin->frameCount = m_accumulationFrames;
}
```

카메라가 조금이라도 움직이면 기존에 누적된 광선 데이터는 잔상(Ghosting)을 일으키므로 `m_accumulationFrames = 0;`으로 리셋합니다. 카메라의 뷰/투영 행렬의 역행렬을 구하고, HLSL 셰이더의 열 우선(Column-major) 행렬 곱셈 규격에 맞추어 `XMMatrixTranspose`로 전치하여 매핑된 메모리에 직접 씁니다.

11.2 Render 및 다단계 자원 장벽 (Resource Barrier)

src/DX12Renderer.cpp (Render & 리소스 배리어)

```
{
    ThrowIfFailed(m_commandAllocator->Reset());
    ThrowIfFailed(m_commandList->Reset(m_commandAllocator.Get(), nullptr));

    ID3D12DescriptorHeap* descriptorHeaps[] = { m_srvUavHeap.Get() };
    m_commandList->SetDescriptorHeaps(_countof(descriptorHeaps), descriptorHeaps);

    m_commandList->SetComputeRootSignature(m_rootSignature.Get());

    // Bind UAV (Render Target)
    m_commandList->SetComputeRootDescriptorTable(0, m_srvUavHeap->GetGPUDescriptorHandleForHeapStart
());

    // Bind TLAS (SRV)
    m_commandList->SetComputeRootShaderResourceView(1, m_topLevelAS->GetGPUVirtualAddress());

    // Bind CBV (b0) -> Parameter 4
    m_commandList->SetComputeRootConstantBufferView(4, m_constantBuffer->GetGPUVirtualAddress());

    // Bind Vertices (t1) -> Parameter 2
    m_commandList->SetComputeRootShaderResourceView(2, m_vertexBuffer->GetGPUVirtualAddress());

    // Bind Indices (t2) -> Parameter 3
    m_commandList->SetComputeRootShaderResourceView(3, m_indexBuffer->GetGPUVirtualAddress());

    // Transition raytracing output to UNORDERED_ACCESS and synchronize accumulation buffer
    D3D12_RESOURCE_BARRIER preRaytraceBarriers[2] = {};
    preRaytraceBarriers[0].Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    preRaytraceBarriers[0].Transition.pResource = m_raytracingOutput.Get();
    preRaytraceBarriers[0].Transition.StateBefore = D3D12_RESOURCE_STATE_COPY_SOURCE;
    preRaytraceBarriers[0].Transition.StateAfter = D3D12_RESOURCE_STATE_UNORDERED_ACCESS;

    preRaytraceBarriers[1].Type = D3D12_RESOURCE_BARRIER_TYPE_UAV;
    preRaytraceBarriers[1].UAV.pResource = m_accumulationBuffer.Get();
    m_commandList->ResourceBarrier(2, preRaytraceBarriers);

    m_commandList->SetPipelineState1(m_dxrStateObject.Get());

    D3D12_DISPATCH_RAYS_DESC dispatchDesc = {};
    dispatchDesc.HitGroupTable.StartAddress = m_shaderBindingTable->GetGPUVirtualAddress() + m_sbtEntry
Size * 2;
    dispatchDesc.HitGroupTable.SizeInBytes = m_sbtEntrySize;
    dispatchDesc.HitGroupTable.StrideInBytes = m_sbtEntrySize;

    dispatchDesc.MissShaderTable.StartAddress = m_shaderBindingTable->GetGPUVirtualAddress() + m_sbtEnt
rySize;
    dispatchDesc.MissShaderTable.SizeInBytes = m_sbtEntrySize;
    dispatchDesc.MissShaderTable.StrideInBytes = m_sbtEntrySize;

    dispatchDesc.RayGenerationShaderRecord.StartAddress = m_shaderBindingTable->GetGPUVirtualAddress();
    dispatchDesc.RayGenerationShaderRecord.SizeInBytes = m_sbtEntrySize;

    dispatchDesc.Width = m_width;
    dispatchDesc.Height = m_height;
```

```
    dispatchDesc.Depth = 1;

    m_commandList->DispatchRays(&dispatchDesc);

    // Transition UAV to COPY_SOURCE and Swapchain to COPY_DEST
    D3D12_RESOURCE_BARRIER copyBarriers[2] = {};
    copyBarriers[0].Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    copyBarriers[0].Transition.pResource = m_raytracingOutput.Get();
    copyBarriers[0].Transition.StateBefore = D3D12_RESOURCE_STATE_UNORDERED_ACCESS;
    copyBarriers[0].Transition.StateAfter = D3D12_RESOURCE_STATE_COPY_SOURCE;

    copyBarriers[1].Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    copyBarriers[1].Transition.pResource = m_renderTargets[m_frameIndex].Get();
    copyBarriers[1].Transition.StateBefore = D3D12_RESOURCE_STATE_PRESENT;
    copyBarriers[1].Transition.StateAfter = D3D12_RESOURCE_STATE_COPY_DEST;

    m_commandList->ResourceBarrier(2, copyBarriers);

    m_commandList->CopyResource(m_renderTargets[m_frameIndex].Get(), m_raytracingOutput.Get());

    // Transition Swapchain to PRESENT
    D3D12_RESOURCE_BARRIER presentBarrier = {};
    presentBarrier.Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    presentBarrier.Transition.pResource = m_renderTargets[m_frameIndex].Get();
    presentBarrier.Transition.StateBefore = D3D12_RESOURCE_STATE_COPY_DEST;
    presentBarrier.Transition.StateAfter = D3D12_RESOURCE_STATE_PRESENT;

    m_commandList->ResourceBarrier(1, &presentBarrier);

    ThrowIfFailed(m_commandList->Close());

    ID3D12CommandList* ppCommandLists[] = { m_commandList.Get() };
    m_commandQueue->ExecuteCommandLists(_countof(ppCommandLists), ppCommandLists);

    ThrowIfFailed(m_swapChain->Present(1, 0));

    FlushCommandQueue();
    m_frameIndex = m_swapChain->GetCurrentBackBufferIndex();
}
```

```
[프레임 렌더링 자원 전이 및 흐름도]
1. m_raytracingOutput: [COPY_SOURCE] ---- Transition Barrier ----> [UNORDERED_ACCESS (UAV)]
2. DispatchRays 실행 (광선 연산 수행)
3. m_raytracingOutput: [UNORDERED_ACCESS] ---- Transition Barrier ----> [COPY_SOURCE]
   스왑체인 백버퍼:   [PRESENT]          ---- Transition Barrier ----> [COPY_DEST]
4. CopyResource (UAV 출력물 -> 백버퍼 고속 복사)
5. 스왑체인 백버퍼:   [COPY_DEST]        ---- Transition Barrier ----> [PRESENT]
6. Present(1, 0) 호출 (화면 주사율에 맞춰 송출)
```

11.3 펜스(Fence)를 통한 GPU 플러시 (FlushCommandQueue)

src/DX12Renderer.cpp (FlushCommandQueue)

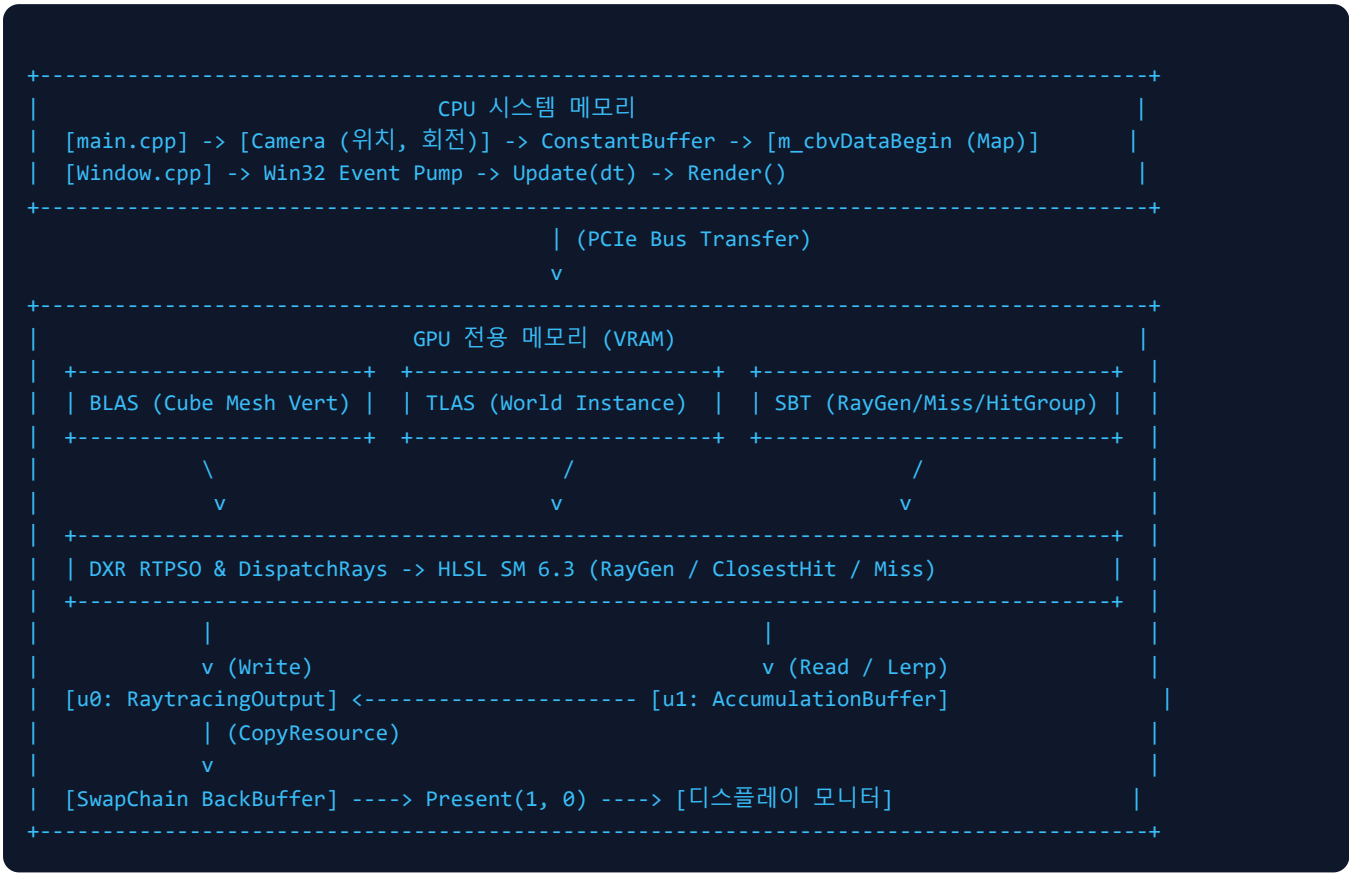
```
void DX12Renderer::FlushCommandQueue()
{
    const UINT64 fenceToWaitFor = m_fenceValue;
    ThrowIfFailed(m_commandQueue->Signal(m_fence.Get(), fenceToWaitFor));
    m_fenceValue++;

    if (m_fence->GetCompletedValue() < fenceToWaitFor)
    {
        ThrowIfFailed(m_fence->SetEventOnCompletion(fenceToWaitFor, m_fenceEvent));
        WaitForSingleObject(m_fenceEvent, INFINITE);
    }
}
```

`m_commandQueue->Signal(m_fence.Get(), fenceToWaitFor)`는 GPU 명령 큐의 가장 마지막 작업 뒤에 펜스 카운터를 증가시키는 마커를 삽입합니다. CPU는 `WaitForSingleObject`로 GPU가 해당 지점에 도달할 때까지 안전하게 대기하여 레이스 컨디션과 메모리 누수를 방지합니다.

Part 12 아키텍처 다이어그램 및 DX12 트러블슈팅 가이드

12.1 전체 시스템 메모리 맵 및 데이터 파이프라인



12.2 DirectX 12 개발자가 자주 겪는 실수 TOP 5 & 해결책

오류 현상	근본 원인	검증 및 해결 방법
DEVICE_REMOVED (TDR 크래시)	HLSL 무한 루프, 잘못된 메모리 주소 역참조, 또는 `RayPayload` 크기 불일치.	디버그 레이어 메시지 확인 및 `MaxPayloadSizeInBytes`와 HLSL 구조체 바이트 크기 일치 검증.
화면이 검은색으로만 나옴	리소스 장벽 누락으로 백버퍼에 복사되지 않았거나, 카메라 역행렬의 전치(Transpose) 누락.	UAV -> `COPY_SOURCE` 및 `COPY_DEST` -> `PRESENT` 전이 확인, `XMMatrixTranspose` 호출 확인.
상수 버퍼 바인딩 크래시	상수 버퍼 크기가 256바이트 배수로 정렬되지 않음.	`(sizeof(T) + 255) & ~255` 매크로를 사용하여 256바이트 패딩 보장.
SBT 바인딩 에러	레코드 크기가 32바이트 또는 테이블 시작 주소가 64바이트로 정렬되지 않음.	`D3D12_RAYTRACING_SHADER_TABLE_BYTE_ALIGNMENT` (64)로 올림 정렬.
간헐적 화면 떨림 / 티어링	스왑 체인 플립 전후 CPU와 GPU 간의 동기화 부재.	`FlushCommandQueue()` 또는 프레임별 펜스를 사용하여 명령 완료 대기.